

RELATÓRIO TÉCNICO – SISTEMA DE GESTÃO DE NOTAS

Disciplina: Estrutura de Dados

Faculdade: Faculdade Municipal de Palhoça (FMP)

Curso: Análise e Desenvolvimento de Sistemas

Data: 24 de setembro de 2026

Equipe: Joel Nunes Varela e Danielly Cardoso do Amaral

1. Organização da Equipe e Responsabilidades:

Integrante	Papéis Operacionais	Principais Atribuições
Joel Nunes Varela	Documentador e Testador do funcionamento	Elaboração da documentação técnica. e condução dos testes funcionais.
Danielly Cardoso do Amaral	Desenvolvedora Principal	Desenvolvimento das rotinas recursivas, funções de cálculo de média, Implementação da estrutura de navegação por menus e rotinas de busca sequencial.

2. Estrutura do Código e Análise Algorítmica

O sistema foi modularizado em funções para garantir a manutenibilidade e a clareza do fluxo de dados:

2.1 Módulos do Sistema

- **cadastrarAluno:** Recebe o nome e 3 notas por aluno, validando a faixa aceita de 0 à 10 antes de realizar a inserção nas listas paralelas.
- **calcularMedia:** Percorre a lista de notas para determinar a média geral da turma de forma iterativa.
- **listarAprovados:** Identifica os discentes que atingiram média igual ou maior que 7.
- **contarAprovadosRecursivo:** Determina o total de alunos aprovados aplicando o conceito de recursividade.
- **somarNotasRecursiva:** Acumula a soma total de todas as notas cadastradas por meio de chamadas recursivas.
- **buscaSequencial:** Localiza registros de alunos pelo nome fornecido pelo usuário.

2.2 Demonstração Teórica das Funções Recursivas

A. Somatório Recursivo de Notas

- **Caso Base:** Quando o índice atinge o tamanho total da lista de notas, a função encerra retornando 0.
- **Passo Redutor:** Soma as notas do aluno atual e chama a si mesma para o próximo índice ($i = i + 1$).
$$\text{Caso Base: } i = \text{len}(\text{notas}) \implies 0$$
$$\text{Passo Redutor: } \sum(\text{notas}[i]) + \text{somarNotasRecursiva}(\text{notas}, i + 1)$$

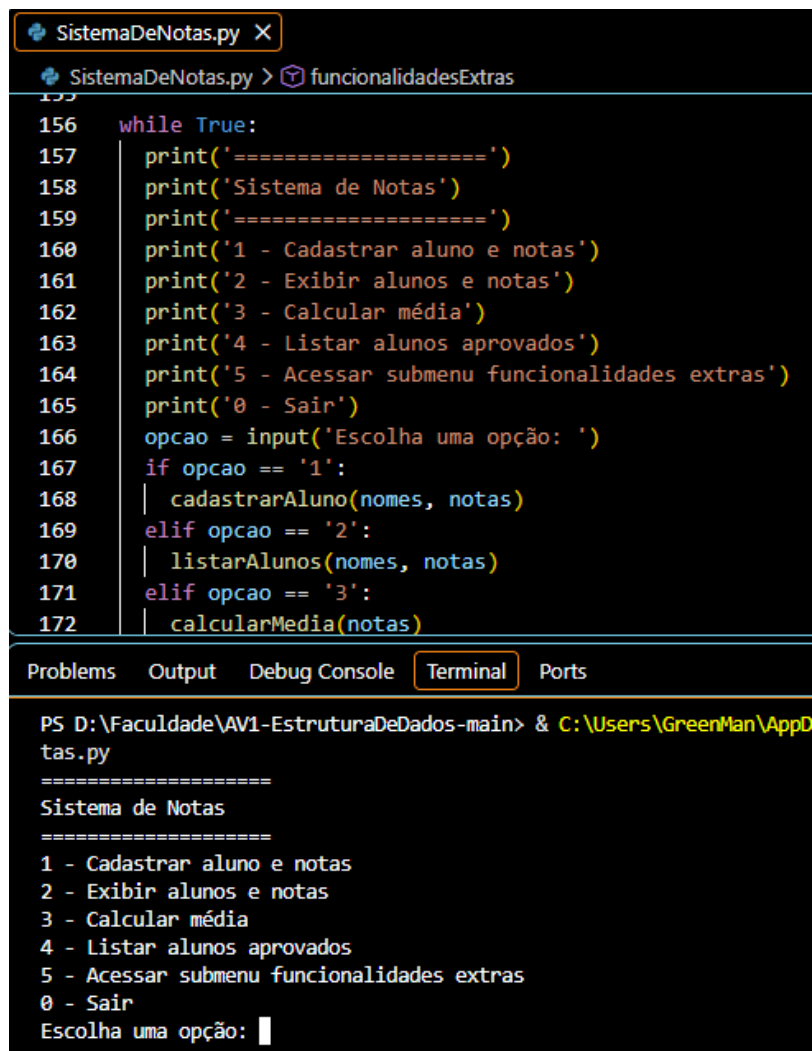
B. Contagem Recursiva de Aprovados

- **Caso Base:** Quando o índice atinge o limite do vetor ($i = \text{len}(\text{notas})$), retorna 0.
- **Passo Redutor:** Verifica se a média das notas do aluno no índice atual é ≥ 7.0 . Se for, soma 1 ao resultado da chamada do próximo elemento; caso contrário, soma 0.

3. Evidências de Execução e Testes de Mesa de Python

Abaixo estão postas as capturas de tela demonstrando a execução das funcionalidades do sistema desenvolvidas em Python.

3.1 Inicialização e Menu Principal

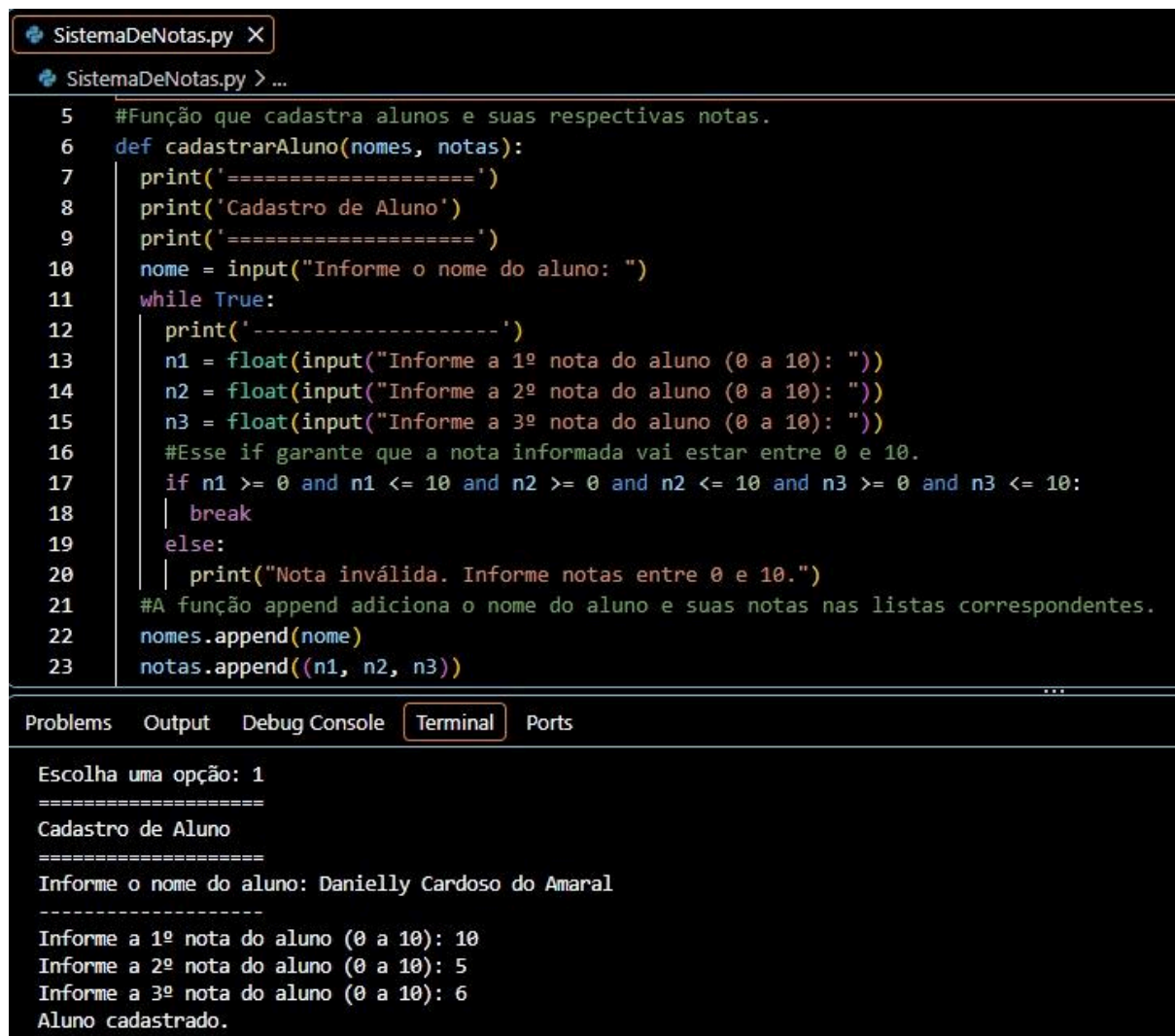


```
SistemaDeNotas.py X
SistemaDeNotas.py > funcionalidadesExtras
156 while True:
157     print('=====')
158     print('Sistema de Notas')
159     print('=====')
160     print('1 - Cadastrar aluno e notas')
161     print('2 - Exibir alunos e notas')
162     print('3 - Calcular média')
163     print('4 - Listar alunos aprovados')
164     print('5 - Acessar submenu funcionalidades extras')
165     print('0 - Sair')
166     opcao = input('Escolha uma opção: ')
167     if opcao == '1':
168         | cadastrarAluno(nomes, notas)
169     elif opcao == '2':
170         | listarAlunos(nomes, notas)
171     elif opcao == '3':
172         | calcularMedia(notas)

Problems Output Debug Console Terminal Ports
PS D:\Faculdade\AV1-EstruturaDeDados-main> & C:\Users\GreenMan\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10.0.0\python.exe SistemaDeNotas.py
=====
Sistema de Notas
=====
1 - Cadastrar aluno e notas
2 - Exibir alunos e notas
3 - Calcular média
4 - Listar alunos aprovados
5 - Acessar submenu funcionalidades extras
0 - Sair
Escolha uma opção: |
```

Figura 1 – Inicialização do Sistema de Notas e exibição do Menu Principal.

3.2 Cadastro de Alunos e Validação de Entradas



```
SistemaDeNotas.py X
SistemaDeNotas.py > ...

5 #Função que cadastra alunos e suas respectivas notas.
6 def cadastrarAluno(nomes, notas):
7     print('=====')
8     print('Cadastro de Aluno')
9     print('=====')
10    nome = input("Informe o nome do aluno: ")
11    while True:
12        print('-----')
13        n1 = float(input("Informe a 1ª nota do aluno (0 a 10): "))
14        n2 = float(input("Informe a 2ª nota do aluno (0 a 10): "))
15        n3 = float(input("Informe a 3ª nota do aluno (0 a 10): "))
16        #Esse if garante que a nota informada vai estar entre 0 e 10.
17        if n1 >= 0 and n1 <= 10 and n2 >= 0 and n2 <= 10 and n3 >= 0 and n3 <= 10:
18            break
19        else:
20            print("Nota inválida. Informe notas entre 0 e 10.")
21    #A função append adiciona o nome do aluno e suas notas nas listas correspondentes.
22    nomes.append(nome)
23    notas.append((n1, n2, n3))
...
```

Problems Output Debug Console **Terminal** Ports

```
Escolha uma opção: 1
=====
Cadastro de Aluno
=====
Informe o nome do aluno: Danielly Cardoso do Amaral
-----
Informe a 1ª nota do aluno (0 a 10): 10
Informe a 2ª nota do aluno (0 a 10): 5
Informe a 3ª nota do aluno (0 a 10): 6
Aluno cadastrado.
```

Figura 2 – Processo de cadastro e validação das 3 notas do discente.

3.3 Processamento Interativo: Exibir alunos e notas.

```
27 def listarAlunos(nomes, notas):
28     if len(nomes) == 0:
29         print("Nenhum aluno cadastrado.")
30         return
31     print('=====')
32     print("Alunos cadastrados:")
33     print('=====')
34     #Percorre a lista de nomes e notas, exibindo-os na tela
35     for i in range(len(nomes)):
36         print(f"{i + 1}. {nomes[i]} - Nota: {notas[i]}")
37
38     #função que calcula a média da turma.
39     def calcularMedia(notas):
40         if len(notas) == 0:
41             return 0
42         return sum(notas) / len(notas)
43
44     #Exibe a média da turma
45     media = calcularMedia(notas)
46     print(f"Média da turma: {media}")
47
48     return
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
3 - Calcular média
4 - Listar alunos aprovados
5 - Acessar submenu funcionalidades extras
6 - Sair
Escolha uma opção: 2
=====
Alunos cadastrados:
=====
1. danielly cardoso - Nota: (9.0, 7.0, 6.0)
=====
```

Figura 3 – Cálculo iterativo da média geral da turma.

```
38  #função que calcula a média da turma.
39  def calcularMedia(notas):
40      if len(notas) == 0:
41          print("Nenhuma nota registrada.")
42          return 0
43      soma = 0
44      quantidadeNotas = 0
45      for notasAluno in notas:
46          for nota in notasAluno:
47              soma += nota
48              quantidadeNotas += 1
49      #Armazena a soma das notas e a quantidade de notas par
50      media = soma / quantidadeNotas
51      print('-----')
52      print(f"A média da turma é: {media:.2f}")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
1 - Cadastrar aluno e notas
2 - Exibir alunos e notas
3 - Calcular média
4 - Listar alunos aprovados
5 - Acessar submenu funcionalidades extras
0 - Sair
Escolha uma opção: 3
-----
A média da turma é: 7.33
=====
```

Figura 4 – Listagem dos alunos aprovados com média superior ou igual a 7.0.

```

56 def listarAprovados(nomes, notas):
57     if len(nomes) == 0:
58         print("Nenhum aluno cadastrado.")
59         return
60     quantidade = 0
61     print('=====')
62     print("Alunos aprovados:")
63     print('=====')
64     #Percorre a lista, utiliza a função sum() para somar todas
divide pelo numero de notas registradas.
65     for i in range(len(nomes)):
66         media = sum(notas[i]) / len(notas[i])
67         if media >= 7:
68             print(f"{nomes[i]}")
69             quantidade += 1
70     if quantidade == 0:

```

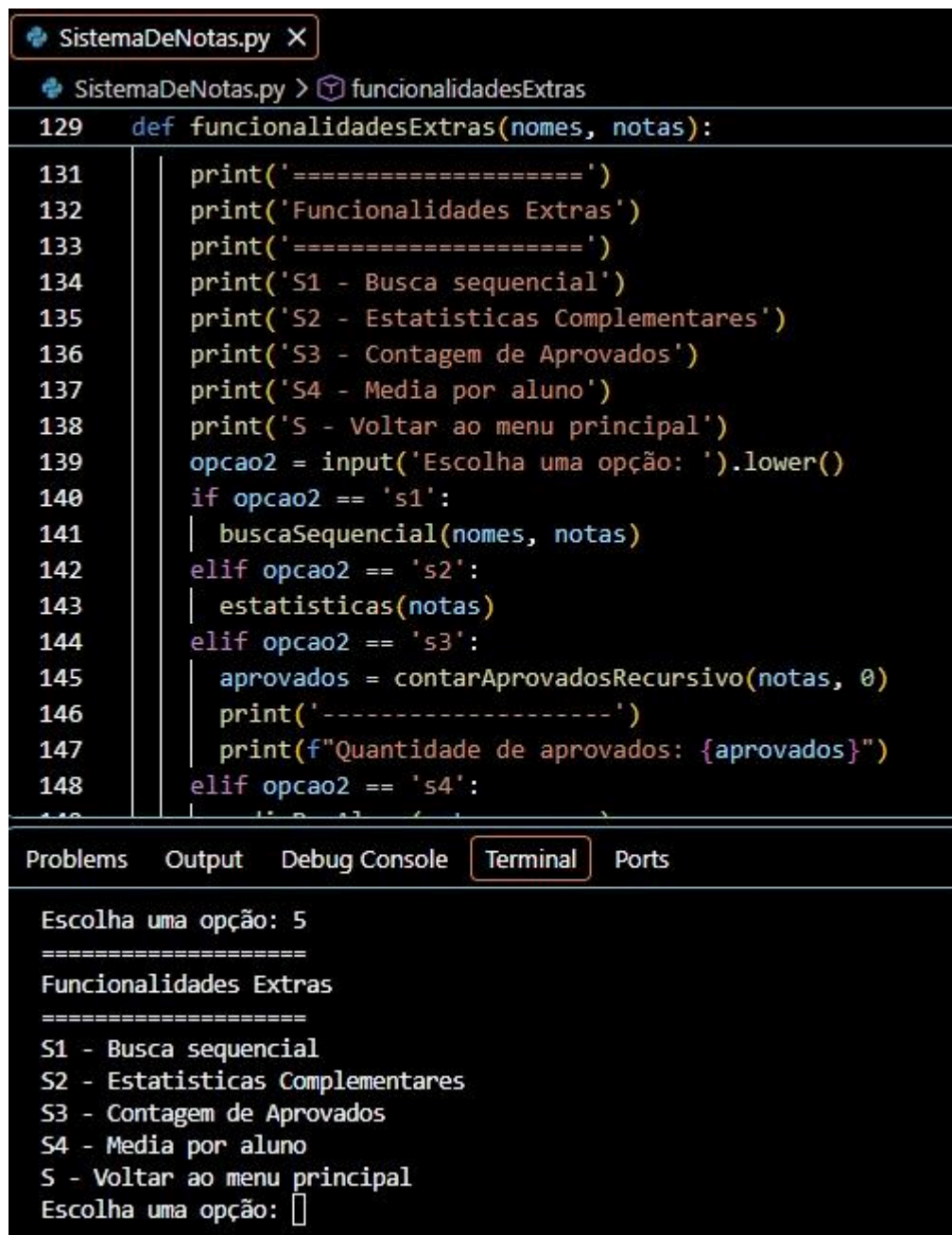
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

=====
1 - Cadastrar aluno e notas
2 - Exibir alunos e notas
3 - Calcular média
4 - Listar alunos aprovados
5 - Acessar submenu funcionalidades extras
0 - Sair
Escolha uma opção: 4
=====
Alunos aprovados:
=====
danielly cardoso
=====

```

3.4 Submenu de Funcionalidades Extras



The image shows a code editor window titled 'SistemaDeNotas.py' with a tab for 'funcionalidadesExtras'. The code defines a function 'funcionalidadesExtras(nomes, notas)' that prints a menu of extra features and handles user input. The menu options are: S1 - Busca sequencial, S2 - Estatísticas Complementares, S3 - Contagem de Aprovados, S4 - Média por aluno, and S - Voltar ao menu principal. The code uses a loop to repeatedly show the menu until the user chooses to return to the main menu. Below the code editor, the 'Terminal' tab is active, showing the program's output. It displays the menu options and the user's input '5', which corresponds to option S.

```
129 def funcionalidadesExtras(nomes, notas):
131     print('=====')
132     print('Funcionalidades Extras')
133     print('=====')
134     print('S1 - Busca sequencial')
135     print('S2 - Estatísticas Complementares')
136     print('S3 - Contagem de Aprovados')
137     print('S4 - Média por aluno')
138     print('S - Voltar ao menu principal')
139     opcao2 = input('Escolha uma opção: ').lower()
140     if opcao2 == 's1':
141         buscaSequencial(nomes, notas)
142     elif opcao2 == 's2':
143         estatisticas(notas)
144     elif opcao2 == 's3':
145         aprovados = contarAprovadosRecursivo(notas, 0)
146         print('-----')
147         print(f"Quantidade de aprovados: {aprovados}")
148     elif opcao2 == 's4':
149         mediaPorAluno(nomes, notas)
150     elif opcao2 == 's':
151         return
152     else:
153         print('Opção inválida. Tente novamente.')
154     funcionalidadesExtras(nomes, notas)
```

Problems Output Debug Console **Terminal** Ports

```
Escolha uma opção: 5
=====
Funcionalidades Extras
=====
S1 - Busca sequencial
S2 - Estatísticas Complementares
S3 - Contagem de Aprovados
S4 - Média por aluno
S - Voltar ao menu principal
Escolha uma opção: 
```

Figura 5 – Interface do submenu de funcionalidades complementares.

3.5 Testes dos Módulos Específicos

Figura 6 – Execução da busca sequencial por nome do aluno.

```
81
82  def buscaSequencial(nomes, notas):
83      print('-----')
84      nomeProcurado = input("Informe o nome do aluno que deseja buscar: ")
85      print('-----')
86      for i in range(len(nomes)):
87          if nomes[i].lower() == nomeProcurado.lower():
88              print('=====')
89              print(f"Aluno encontrado: {nomes[i]} - Nota: {notas[i]}")
90              return i
91      print("Aluno não encontrado.")
92      return -1
93
94  def somarNotasRecursiva(notas, indice):
95      if indice == len(notas):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Funcionalidades Extras
=====

51 - Busca sequencial
52 - Estatísticas Complementares
53 - Contagem de Aprovados
54 - Média por aluno
5 - Voltar ao menu principal
Escolha uma opção: 51

Informe o nome do aluno que deseja buscar: danielly cardoso

=====
Aluno encontrado: danielly cardoso - Nota: (9.0, 7.0, 6.0)

Figura 7 – Exibição do relatório de estatísticas complementares.

```

99
100 def estatisticas(notas):
101     if len(notas) == 0:
102         print('Nenhuma nota cadastrada.')
103         return
104     soma = somarNotasRecursiva(notas, 0)
105     aprovados = contarAprovadosRecursivo(notas, 0)
106     quantidadeNotas = len(notas) * 3
107     media = soma / quantidadeNotas
108     percentual = (aprovados / len(notas)) * 100
109     print('=====')
110     print("Estatísticas Complementares:")
111     print('=====')
112     print(f"Soma das notas: {soma:.2f}")
113     print(f"Média da turma: {media:.2f}")
114     print(f"Quantidade de aprovados: {aprovados}")
115     print(f"Percentual de aprovação: {percentual:.2f}%")
116

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

S4 - Média por aluno
S - Voltar ao menu principal
Escolha uma opção: s2
=====
Estatísticas Complementares:
=====
Soma das notas: 22.00
Média da turma: 7.33
Quantidade de aprovados: 1
Percentual de aprovação: 100.00%

```

Figura 8 – Resultado da contagem recursiva de alunos aprovados.

```

73  #Função que conta a quantidade de alunos aprovados de forma recur
74  def contarAprovadosRecursivo(notas, indice):
75      if indice == len(notas):
76          return 0
77      mediaAluno = sum(notas[indice]) / len(notas[indice])
78      if mediaAluno >= 7:
79          return 1 + contarAprovadosRecursivo(notas, indice + 1)
80      return contarAprovadosRecursivo(notas, indice + 1)
81

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Funcionalidades Extras

=====

S1 - Busca sequencial

S2 - Estatísticas Complementares

S3 - Contagem de Aprovados

S4 - Média por aluno

S - Voltar ao menu principal

Escolha uma opção: s3

Quantidade de aprovados: 1

Figura 9 – Listagem individualizada das médias obtidas por cada aluno.

```

118 def mediaPorAluno(notas, nomes):
119     medias = []
120     for nota in notas:
121         media = sum(nota) / len(nota)
122         medias.append(media)
123     print('-----')
124     print("Médias dos alunos:")
125     print('-----')
126     for i in range(len(medias)):
127         print(f"{i + 1}. {nomes[i]} - Média: {medias[i]:.2f}")
128
129 def funcionalidadesExtras(nomes, notas):
130     while True:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

S1 - Busca sequencial
S2 - Estatísticas Complementares
S3 - Contagem de Aprovados
S4 - Media por aluno
S - Voltar ao menu principal
Escolha uma opção: s4
-----
Médias dos alunos:
-----
1. danielly cardoso - Média: 7.33
=====

```

4. Testes de Execução e Validação em Linguagem C

Abaixo estão reservadas as seções para inclusão das capturas de tela da versão em linguagem C, validando a paridade de execução em relação ao protótipo desenvolvido em Python.

4.1 Inicialização e Menu Principal (Linguagem C)

```
3 int main ( ) {
306 char nomes [ MAX_ALUNOS ] [ 100 ] ;
2 float notas [ MAX_ALUNOS ] [ NUM_NOTAS ] ;
4 int ;
1 enquanto ( 1 ) {
5 printf ( " ===== \n " ) ;
7 printf ( " Sistema de Notas \n " ) ;
8 printf ( " ===== \n " ) ;
9
10 printf ( " 1-Cadastro de alunos e notas \n " ) ;
11 printf ( " 2-Exibir alunos e notas \n " ) ;
12 printf ( " 3-Calcular média \n " ) ;
13 printf ( " 4-Listar alunos aprovados \n " ) ;
14 printf ( " 5-Acessar funcionalidades extras do submenu \n " ) ;
17 printf ( " 0 - Sair \n " ) ;
16
15 int opcao ;
303 printf ( " Escolha uma opção: " ) ;
304

=====
Sistema de Notas
=====
1 - Cadastrar aluno e notas
2 - Exibir alunos e notas
3 - Calcular media
4 - Listar alunos aprovados
5 - Acessar submenu funcionalidades extras
0 - Sair
Escolha uma opção: 
```

Figura 11 – Inicialização do programa compilado em C e apresentação das opções do Menu Principal.

4.2 Cadastro de Alunos e Média Individual (Linguagem C)

```

7~ void cadastrarAluno ( char nomes [ ] [ MAX_ALUNOS ] , float notas [ ] [ NUM_NOTAS ] , int * quantidade ) {
1   printf ( " ===== \n " ) ;
11  printf ( " Cadastro de Aluno \n " ) ;
10  printf ( " ===== \n " ) ;
4   printf ( " Digite o nome do aluno: " ) ;
14  scanf ( " %[\n] " , nomes [ * quantidade ] ) ;
13
9   //Loop repete até que o usuário insira notas válidas (entre 0 e 10) para as três notas do aluno.
12~ enquanto ( 1 ) {
5   printf ( " ----- \n " ) ;
8
2   printf ( " Informe a 1ª nota do aluno: " ) ;
6   scanf ( " %f " , & notas [ * quantidade ] [ 0 ] ) ;
3
16  printf ( " Informe a 2ª nota do aluno: " ) ;
17  scanf ( " %f " , & notas [ * quantidade ] [ 1 ] ) ;
303
15  printf ( " Informe a 3ª nota do aluno: " ) ;
306

```

5 - Acessar submenu funcionalidades extras
0 - Sair
Escolha uma opcao: 1
=====

Cadastro de Aluno
=====

Digite o nome do aluno: Joel Nunes Varela
=====

Informe a 1ª nota do aluno: 8.5
Informe a 2ª nota do aluno: 7
Informe a 3ª nota do aluno: 6
Aluno cadastrado com sucesso!

Ativar o W
Acesse Config

```

305  if ( notas [ * quantidade ] [ 0 ] >= 0 &&
7      notas [ * quantidade ] [ 0 ] <= 10 &&
1      notas [ * quantidade ] [ 1 ] >= 0 &&
10     notas [ * quantidade ] [ 1 ] <= 10 &&
11     notas [ * quantidade ] [ 2 ] >= 0 &&
4~    notas [ * quantidade ] [ 2 ] <= 10 ) {
14
13     quebrar ;
9     //faz a verificação se as notas estão dentro do intervalo válido (0
12~ } outro {
8     printf ( " Notas inválidas. Digite novamente. \n " ) ;
5     }
2     }
6     ( * quantidade ) ++ ;
3     //incrementa a quantidade de alunos cadastrados.
16     printf ( " Aluno cadastrado com sucesso! \n " ) ;
17 }
303

```

0 - Sair
Escolha uma opcao: 1
=====

Cadastro de Aluno
=====

Digite o nome do aluno: Danielly Cardoso do Amaral
=====

Informe a 1ª nota do aluno: 10
Informe a 2ª nota do aluno: 5
Informe a 3ª nota do aluno: 6
Aluno cadastrado com sucesso!
=====

Figuras 12 e 13 – Cadastro dos alunos com a validação das notas no terminal em C.

4.3 Processamento Iterativo: Média da Turma e Aprovados (Linguagem C)

```
304 //Soma a as notas de todos os alunos e calcula a média da turma.
307 float soma = 0;
1 int Notas = 0;
7 for ( int i = 0; i < quantidade; i++) {
284 for ( int j = 0; j < NUM_NOTAS; j++) {
9 soma += notas [ i ] [ j ];
3 Notas ++;
10 }
11 }
5
4 média flutuante = soma / *Notas;
14 printf ( " ----- \n " );
17 printf ( " A média da turma é: %.2f\n ", média );
305 média de retorno;
13 }
12
8
6 //função para listar os alunos aprovados (média >= 7).
16 void listarAprovados ( char nomes [ ] [ MAX_ALUNOS ], float notas [ ] [ NUM_NOTAS ], int quantidade ) {
306 se ( quantidade == 0 ) {
303 printf ( " Nenhum aluno cadastrado. \n " );
285 }
}

Escolha uma opcao: 3
-----
A média da turma é: 7.08
-----
Sistema de Notas
-----
1 - Cadastrar aluno e notas
```

Figura 13 – Processamento iterativo e exibição da média geral da turma em C (Resultado: 7.08).

```
282 printf ( " Alunos aprovados: \n " );
15 printf ( " ===== \n " );
307 //percorre o vetor de nomes e notas, calculando a média de cada aluno. Se a média
304 int Aprovados = 0;
2 for ( int i = 0; i < quantidade; i++) {
9 float soma = 0;
284 for ( int j = 0; j < NUM_NOTAS; j++) {
7 soma += notas [ i ] [ j ];
11 }
10 float media = soma / NUM_NOTAS;
3 se ( media >= 7 ) {
1 printf ( " %d . %s - Média: %.2f\n ", i + 1, nomes [ i ], media );
14 QuantidadeAprovados ++;
4 }
5 }
13
305 if ( quantidadeAprovados == 0 ) {
17 printf ( " Nenhum aluno aprovado. \n " );
6 }
8 }
12
303
286

Escolha uma opcao: 4
-----
Alunos aprovados:
-----
1. Joel Nunes Varela - Média: 7.17
2. Danielly Cardoso do Amaral - Média: 7.00
-----
```

Figura 14 – Exibição dos alunos com média ≥ 7.0 identificados na execução em C.

4.4 Submenu de Funcionalidades Extras (Linguagem C)

```
10 //função para exibir um submenu com funcionalidades extras.
305 void funcionalidadesExtras ( char nomes [ ] [ 100 ], float notas [ ] [ NUM_NOTAS ], int quantidade ) {
8 char opcao2 [ 10 ];
306 enquanto ( 1 ) {
5 printf ( " ===== \n " );
284 printf ( " Funcionalidades Extras \n " );
6 printf ( " ===== \n " );
14
17 printf ( " S1-Busca sequencial \n " );
11 printf ( " S2-Estatísticas Complementares \n " );
282 printf ( " S3-Contagem de Aprovados \n " );
303 printf ( " S4-Media por aluno \n " );
12 printf ( " S-Voltar ao menu principal \n " );
285
307 printf ( " Escolha uma opção: " );
304 scanf ( " %s ", opcao2 );
3
9 //converte a opção digitada para minúsculas para facilitar a comparação.
4 para ( int i = 0 ; opcao2 [ i ] != '\0' ; i ++ ) {
15 opcao2 [ i ] = tolower ( opcao2 [ i ] );
7
}

=====
Funcionalidades Extras
=====
S1 - Busca sequencial
S2 - Estatísticas Complementares
S3 - Contagem de Aprovados
S4 - Media por aluno
S - Voltar ao menu principal
Escolha uma opcao:

Ativar o
Acesse Con
```

Figura 15 – Interface e navegação do submenu de estatísticas complementares no programa em C.

5.5 Teste das Funções Recursivas e Busca (Linguagem C)

```
304 soma += notas [ indice ] [ j ];
13 }
305 float mediaAluno = soma / NUM_NOTAS ;
2 se ( mediaAluno >= 7 ) {
306 return 1 + contarAprovadosRecursivo ( notas , quantidade , índice + 1 );
10 } outro {
286 return contarAprovadosRecursivo ( notas , quantidade , índice + 1 );
285 }
308 } //A função percorre o vetor de notas recursivamente, calculando a média de cada aluno e contando quantos estão aprovados.
7
1
12 //função para buscar um aluno pelo nome usando busca sequencial.
4 int buscaSequencial ( char nomes [ ] [ MAX_ALUNOS ], float notas [ ] [ NUM_NOTAS ], int quantidade ) {
3 char nomeBusca [ MAX_ALUNOS ];
6 printf ( " ===== \n " );
8 printf ( " Digite o nome do aluno que deseja buscar: " );
5
307 // CORREÇÃO: Usar " %[^\\n] " para ler o nome completo com espaços e descartar o Enter pendente
283 scanf ( " %[^\\n] ", nomeBusca );
16
11 printf ( " ===== \n " );
15 for ( int i = 0 ; i < quantidade ; i ++ ) {
17
}

=====
Digite o nome do aluno que deseja buscar: Joel Nunes Varela
=====
Aluno encontrado: Joel Nunes Varela - Notas: 8.50, 7.00, 6.00
=====
Funcionalidades Extras
=====

Ativar o Window
Acesse Configurações
```

Figura 16 – Retorno da busca sequencial por nome implementada na linguagem C.

```

304     soma += notas [ indice ] [ j ] ;
13 }
305 float mediaAluno = soma / NUM_NOTAS ;
2 se ( mediaAluno >= 7 ) {
306     return 1 + contarAprovadosRecursivo ( notas , quantidade , índice + 1 ) ;
10 } outro {
286     return contarAprovadosRecursivo ( notas , quantidade , índice + 1 ) ;
285 }
308 } //A função percorre o vetor de notas recursivamente, calculando a média de cada aluno e contando quantos estão
7
1
12 //função para buscar um aluno pelo nome usando busca sequencial.
4 int buscaSequencial ( char nomes [ ] [ MAX_ALUNOS ] , float notas [ ] [ NUM_NOTAS ] , int quantidade ) {
3 char nomeBusca [ MAX_ALUNOS ] ;
6 printf ( " ----- \n " ) ;
8 printf ( " Digite o nome do aluno que deseja buscar: " ) ;
5
307 // CORREÇÃO: Usar " %[^\n]" para ler o nome completo com espaços e descartar o Enter pendente
283 scanf ( " %[^\n]" , nomeBusca ) ;
16
11 printf ( " ----- \n " ) ;
15 for ( int i = 0 ; i < quantidade ; i ++ ) {
17

```

estatísticas da turma:

```

-----
Soma das notas: 42.50
Média da turma: 7.08
Percentual de alunos aprovados: 100.00%
Quantidade de alunos aprovados: 2
=====

```

Ativar o W
Acesse Config

Figura 17 – Exibição da soma total das notas, média e percentual de aprovação via código C.

```

308 printf ( " ===== \n " ) ;
307 printf ( " Alunos aprovados: \n " ) ;
4 printf ( " ===== \n " ) ;
16 //percorre o vetor de nomes e notas, calculando a média de cada aluno. Se a média for maior ou igual a 7, o aluno é aprovado.
11 int Aprovados = 0 ;
3 for ( int i = 0 ; i < quantidade ; i ++ ) {
12 float soma = 0 ;
9 for ( int j = 0 ; j < NUM_NOTAS ; j ++ ) {
8 soma += notas [ i ] [ j ] ;
17 }
305 float media = soma / NUM_NOTAS ;
13 se ( media >= 7 ) {
286 printf ( " %d . %s - Média: %.2f\n " , i + 1 , nomes [ i ] , media ) ;
10 QuantidadeAprovados ++ ;
306 }
2 }
304
285 if ( quantidadeAprovados == 0 ) {
5 printf ( " Nenhum aluno aprovado. \n " ) ;
1 }
6 }
15

```

estatísticas da turma:

```

-----
S3 - Contagem de Aprovados
S4 - Media por aluno
S - Voltar ao menu principal
Escolha uma opcao: s3
-----
Quantidade de aprovados: 2
=====

```

Figura 18 – Resultado retornado pela função recursiva *contarAprovadosRecursivo* em C (Total: 2 aprovados).

```

8
17 printf ( " ===== \n " );
306 printf ( " Média por aluno: \n " );
3 printf ( " ===== \n " );
13 //percorre o vetor de nomes e notas, calculando a média de cada aluno e exibindo o resultado.
14- for ( int i = 0 ; i < quantidade ; i ++ ) {
285 float soma = 0 ;
304- for ( int j = 0 ; j < NUM_NOTAS ; j ++ ) {
305 soma += notas [ i ] [ j ] ;
2 }
10 float media = soma / NUM_NOTAS ;
286 printf ( " %d.Média do aluno %s : %.2f\n " , i + 1 , nomes [ i ] , media ) ;
4 }
6 }
1
283
15 //função para exibir um submenu com funcionalidades extras.
5- void funcionalidadesExtras ( char nomes [ ] [ 100 ] , float notas [ ] [ NUM_NOTAS ] , int quantidade ) {
12 char opcao2 [ 10 ] ;
308- enquanto ( 1 ) {
307 printf ( " ===== \n " );
9 printf ( " Funcionalidades Extras \n " );
11

```



Figura 19 – Listagem formatada das médias individuais obtidas na versão em C.

5. Comparativo e Paridade de Resultados (Python vs C)

Para evidenciar a equivalência entre os algoritmos construídos nas duas linguagens, a tabela a seguir consolida os valores obtidos nos testes de mesa padronizados:

Funcionalidade / Métrica	Resultado Python	Resultado C	Paridade Confirmada
Média Geral da Turma	\$7.08\$	\$7.08\$	Sim
Alunos Aprovados (Iterativo)	Joel / Danielly	Joel / Danielly	Sim
Contagem Recursiva de Aprovados	\$2\$	\$2\$	Sim
Soma Total das Notas	\$42.50\$	\$42.50\$	Sim
Percentual de Aprovação	\$100.00\%\$	\$100.00\%\$	Sim

6. Conclusão

A implementação atendeu aos requisitos propostos para o trabalho. A utilização de estruturas de listas paralelas permitiu organizar os nomes e notas dos alunos, enquanto as funções recursivas executaram o cálculo acumulado e a contagem de critérios de aprovação sem o uso de laços de repetição tradicionais no corpo das rotinas recursivas.